VS Code Setup for C++, GitHub, and Version Control

1. Install Visual Studio Code (VS Code)

Step 1: Go to the official VS Code website and download the installer for your OS (Windows, macOS, or Linux):

https://code.visualstudio.com/

Step 2: Run the installer:

- Windows: Follow the installation wizard and select the necessary options (e.g., "Add to PATH" for command line usage).
- macOS: Drag and drop the application into the Applications folder.
- Linux: Follow specific distribution instructions available on the site (e.g., using a terminal with sudo apt install code for Ubuntu).

2. Setting Up C++ Development in VS Code

Step 1: Install C++ Compiler

- Windows: Install MinGW or MSYS2 for the g++ compiler.
    - MinGW Setup: Download the MinGW installer and select "mingw32-gcc-g++". http://mingw-w64.org/doku.php/download
    - MSYS2: Use pacman -S mingw-w64-x86_64-gcc to install the compiler. https://www.msys2.org/

- macOS: Install Xcode command line tools by running:

  xcode-select --install

- Linux: Use the package manager to install g++:

  sudo apt install g++

Step 2: Install VS Code Extensions for C++

- Open VS Code, and go to the Extensions view by clicking the square icon on the left sidebar or pressing Ctrl+Shift+X.
- Search for and install the following extensions:
  - C/C++ by Microsoft (for IntelliSense, debugging, etc.)
  - Code Runner (to quickly run code)

Step 3: Configure C++ Environment in VS Code

1. Create a folder for your project.
2. Inside the folder, create a .vscode folder, then create the following files:

tasks.json for build tasks:

```
{
  "version": "2.0.0",
  "tasks": [
      {
          "label": "g++ build active file",
          "command": "/usr/bin/g++", // Or path to g++ on your system
          "args": [
              "-g",
              "${file}",
              "-o",
              "${fileDirname}/${fileBasenameNoExtension}"
```

```
        ],
        "group": {
            "kind": "build",
            "isDefault": true
        },
        "problemMatcher": ["$gcc"],
        "detail": "Generated task by C++ extension"
    }
  ]
}
```

launch.json for debugging:

```
{
  "version": "0.2.0",
  "configurations": [
    {
        "name": "C++ Launch",
        "type": "cppdbg",
        "request": "launch",
        "program": "${fileDirname}/${fileBasenameNoExtension}",
        "args": [],
        "stopAtEntry": false,
        "cwd": "${workspaceFolder}",
        "environment": [],
        "externalConsole": false,
        "MIMode": "gdb",
```

```
        "miDebuggerPath": "/usr/bin/gdb", // Adjust path if needed
        "setupCommands": [
            {
                "description": "Enable pretty-printing for gdb",
                "text": "-enable-pretty-printing",
                "ignoreFailures": true
            }
        ],
        "preLaunchTask": "g++ build active file",
        "internalConsoleOptions": "openOnSessionStart"
    }
  ]
}
```

3. GitHub Setup

Step 1: Create a GitHub Account

1. Go to GitHub: https://github.com/
2. Sign up by filling in your details (username, email, password).
3. After signing up, confirm your email.

Step 2: Set Up Git on Your Local Machine

- Windows: Download and install Git: https://git-scm.com/.
  - During installation, choose the "Use Git from the command line and also from 3rd-party software" option.

- macOS/Linux: Git is typically pre-installed. If not, you can install it using:

  sudo apt install git

  (For macOS, Git comes with Xcode command-line tools.)

Step 3: Configure Git

1. Open a terminal or Git Bash (for Windows).

2. Set your username and email for Git:

   git config --global user.name "Your Name"

   git config --global user.email "youremail@example.com"

4. GitHub: Basic Commands (Repository, Files, Commit)

Step 1: Create a New Repository on GitHub

1. Go to your GitHub profile and click on New Repository.

2. Give it a name and set it to public/private. Optionally, add a README file.

3. Click Create Repository.

Step 2: Initialize Git in Your Local Project

1. Open the terminal inside the VS Code project folder and run:

   git init

2. Add the remote repository to your local Git repository:

   git remote add origin https://github.com/username/repo-name.git

Step 3: Add Files to Git and Make a Commit

1. Add files to Git?s tracking:

   git add .

2. Commit the changes:

```
git commit -m "Initial commit"
```

5. Link GitHub with VS Code and Push to GitHub

Step 1: Install Git Extension in VS Code

1. Open VS Code, go to the Extensions view (Ctrl+Shift+X), and search for GitHub Repositories.

2. Install the GitLens extension for Git support.

Step 2: Link VS Code with Your GitHub Account

1. In VS Code, press Ctrl+Shift+P and type Git: Clone.

2. Enter the URL of the repository you created on GitHub.

3. Once cloned, VS Code will automatically open the project folder.

Step 3: Commit and Push from VS Code

1. Make changes to a file and save it.

2. Open the Source Control panel from the left sidebar (or press Ctrl+Shift+G).

3. You will see your changed files under "Changes". Stage them by clicking the + icon.

4. Write a commit message in the text box at the top and click the checkmark to commit.

5. Click on the ... (more actions) icon in the Source Control panel and choose Push to push the changes to GitHub.